



CodeNicely Shortlisting Process



CodeNicely Software Services LLP



E-commerce Fullstack Developer Assignment (Vue.js + Django)

Objective:

The goal of this assignment is to demonstrate your full-stack development skills by building an e-commerce web application that fetches product data via a Django backend API, stores it in a database, and presents it dynamically using a Vue.js frontend with Vuex for state management. The frontend should include features like product filtering, category-based navigation, search functionality with debouncing, a shopping cart system, and real-time updates using computed properties and watchers.

Assignment Details:

Backend (Django):

1. Setup Django Project:

- Create a Django project and a Django app.
- Configure PostgreSQL (or SQLite) as the database.

2. Database Schema & Models:

○ User Model:

- `mobile` (string) - Unique mobile number.
- `full_name` (string) - Full name of the user.

○ Product Model:

- `title` (string) - The name of the product.
- `price` (float) - The cost of the product.
- `description` (text) - A brief summary of the product.
- `category` (string) - The category to which the product belongs.
- `image_url` (URL field) - The link to the product image.
- `sold` (Boolean) - Indicates whether the product has been sold.
- `is_sale` (Boolean) - Indicates if the product is on sale.
- `date_of_sale` (Date, nullable) - The date the product was sold (if applicable).

- **Cart Management Model:**
 - `user` (ForeignKey to User) - The customer who owns the cart.
 - `product` (ForeignKey to Product) - The product added to the cart.
 - `quantity` (integer) - The quantity of the product in the cart.
- **Order Management Model:**
 - `user` (ForeignKey to User) - The customer who placed the order.
 - `product` (ForeignKey to Product) - The ordered product.
 - `price` (ForeignKey to Product) - The price of the product at the time of order.
 - `quantity` (integer) - The number of units ordered.
 - `payment_mode` (Choice Field) - Payment method (e.g., COD, Online).

3. API Development (Django REST Framework):

- **Authentication Endpoints:**
 - Login with mobile OTP.
- **Product Endpoints:**
 - Fetch all products.
 - Filter products based on category, sold status, and price range.
 - Search products by title.
 - Retrieve product details by ID.
- **Cart Management Endpoints:**
 - Add to cart API.
 - Increase/decrease product quantity in cart.
 - Remove product from cart.
- **Order Management Endpoints:**
 - Create order API.
 - Get order details API.
 - Order cancellation API.

4. Seed Data:

- Populate the database using provided JSON data.

Sample JSON Data:

```
[
  {
    "title": "Wireless Headphones",
    "price": 150.0,
    "description": "High-quality wireless headphones with noise cancellation.",
    "category": "Electronics",
    "image_url": "https://example.com/images/wireless-headphones.jpg",
    "sold": false,
    "is_sale": true
  },
  {
    "title": "Gaming Laptop",
    "price": 1200.0,
    "description": "High-performance laptop for gaming and productivity.",
    "category": "Electronics",
    "image_url": "https://example.com/images/gaming-laptop.jpg",
    "sold": false,
    "is_sale": false
  }
]
```

Frontend (Vue.js + Vuex):

8. Most Bought Products Section:

- Create a section titled "Most Bought Products" on the product listing page.
- Fetch and display the top purchased products based on order quantity.
- Ensure real-time updates when a product is bought.
- Display product details such as title, price, and image.

9. Project Setup:

- Use Vue 3 with Vuex for state management.
- Install and configure Axios for API communication.

10. Product Listing Page:

- Display products in a grid format with details:
 - Title
 - Price
 - Image

- Category
- Sale badge (Green/Red color-coded)
- "Add to Cart" button

11. Filters and Search:

- Implement category-based filtering (Sidebar navigation).
- Implement price range filtering using a slider.
- Implement a search bar with debounced input.

12. Shopping Cart System:

- Users should be able to:
 - Add products to cart.
 - Increase/decrease quantity.
 - Remove products from cart.
 - View cart summary.
 - Proceed to checkout.

13. Order Management:

- Allow users to:
 - Place an order.
 - View past orders with total price, quantity, and date.
 - Cancel orders if allowed.

14. Authentication & User Management:

- Implement login via mobile OTP.
- Store authenticated user details in Vuex.

15. Vue Features Integration:

- **Vuex:**
 - Manage cart, products, and user authentication globally.
 - **Computed Properties:**
 - Dynamically update total price and quantity in the cart.
 - **Watchers:**
 - React to changes in search and filters in real-time.
 - **Debounce Functions:**
 - Optimize search input to reduce API calls.
 - **Global Functions:**
 - Format prices and handle authentication globally.
-

Bonus Features:

- Add a wishlist feature.
 - Enable product reviews and ratings.
 - Implement payment gateway integration.
 - Deploy the app to Heroku/Vercel and provide a live demo link.
-

Submission Guidelines:

1. Provide a GitHub repository with clear setup instructions.
2. Include an `.env` file (if needed) with necessary configurations.
3. Provide a README file with:
 - Project Overview
 - Setup Instructions
 - API Documentation
 - Screenshots of the UI
 - Deployment Link (if applicable)

Best of Luck for your Assignment!